COLLEGE OF COMPUTING AND INFORMATICS

PUTRAJAYA CAMPUS

ASSIGNMENT PART 1

SEMESTER 2 2025/2026

SUBJECT CODE : CSEB5223

SUBJECT : SOFTWARE CONSTRUCTION & METHODS

START DATE : 16 Feb 2026

DUE DATE : 23 Feb 2026

NUMBER OF GROUP : GROUP 3

NAME OF MEMBERS :

No.	Full Name	Student ID
1.	AHMAD ILYAS BIN AHMAD IRFAN	SW01083514
2.	WAN NUR SABRINA BINTI BOKHARI	BSW01085928
3.	NUR HIDAYAH BINTI SARWANI	BSW01085937

Table of Contents

1.0 INTRODUCTION 3
1.1 PROJECT BACKGROUND 3
1.2 PROJECT OBJECTIVE 4
2.0 USE CASE AND CLASS DESIGN OF SLMS 5
2.1 USE CASE DIAGRAM 5
2.2 CLASS DIAGRAM 7
..... 7
3.0 DOCUMENTATION FORMATTING 9
4.0 CODING FILE FORMATTING 10
5.0 NAMING CONVENTIONS 11
6.0 FUNCTION AND METHODS MODULARITY 12
7.0 CONTINUOUS INTEGRATION RULES 13
8.0 REFERENCES 14

1.0 INTRODUCTION

1.1 PROJECT BACKGROUND

With the rapid growth of digital technology in education, learning institutions are increasingly relying on online platforms to manage teaching and learning activities. Traditional classroom-based management methods such as manual assignment submission, physical attendance tracking, and paper-based grading are inefficient and difficult to scale.

A Student Learning Management System (SLMS) is a web-based application designed to support digital learning processes. Similar to established platforms like Moodle, an SLMS enables students, lecturers, and administrators to interact within a centralized system. It allows course management, assignment submission, grading, material distribution, and discussion forums to be handled electronically.

The proposed SLMS will serve as a simplified academic platform that manages users, courses, learning materials, assignments, submissions, and discussions. The system emphasizes proper software construction practices, clean architecture, and structured development to ensure maintainability and scalability.

This project is developed as part of the Software Construction & Maintenance course to apply theoretical knowledge into practical system design and implementation.

1.2 PROJECT OBJECTIVE

The main objective of this project is to design and plan the overall structure of a Student Learning Management System (SLMS) by applying proper software construction principles.

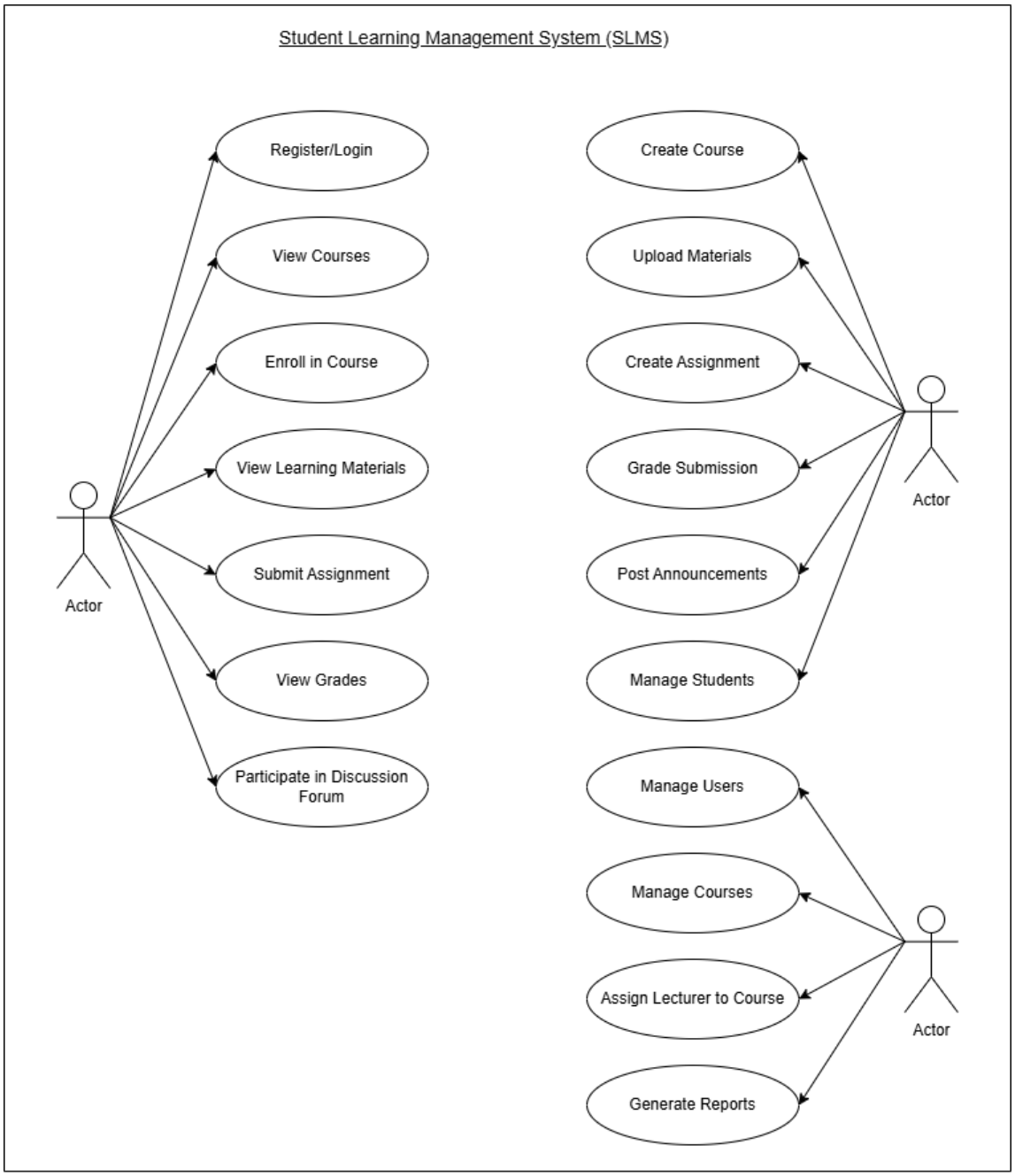
The specific objectives are:

1. To draft the overall system design including use case and class diagrams.
2. To define documentation standards for consistent and professional reporting.
3. To establish coding standards and file formatting rules to ensure clean and maintainable code.
4. To define naming conventions and a terminology dictionary for consistency across development.
5. To apply modularity principles in function and method design.
6. To define continuous integration rules to support collaborative and controlled software development.

Through this project, students will demonstrate their understanding of structured software planning, object-oriented design, and disciplined development practices in building a scalable academic management system.

2.0 USE CASE AND CLASS DESIGN OF SLMS

2.1 USE CASE DIAGRAM



Actors:

1. Student

- Represents learners who access courses, submit assignments, and view academic progress.

2. Lecturer

- Represents instructors who manage courses, upload materials, and assess students.

3. Admin

- Represents system administrators responsible for managing users, system configurations, and course assignments.

Student interactions:

- Registers and logs in to access the system.
- Enrolls in available courses.
- Accesses learning materials uploaded by lecturer.
- Submits assignments before due date.
- Views grades and feedback.

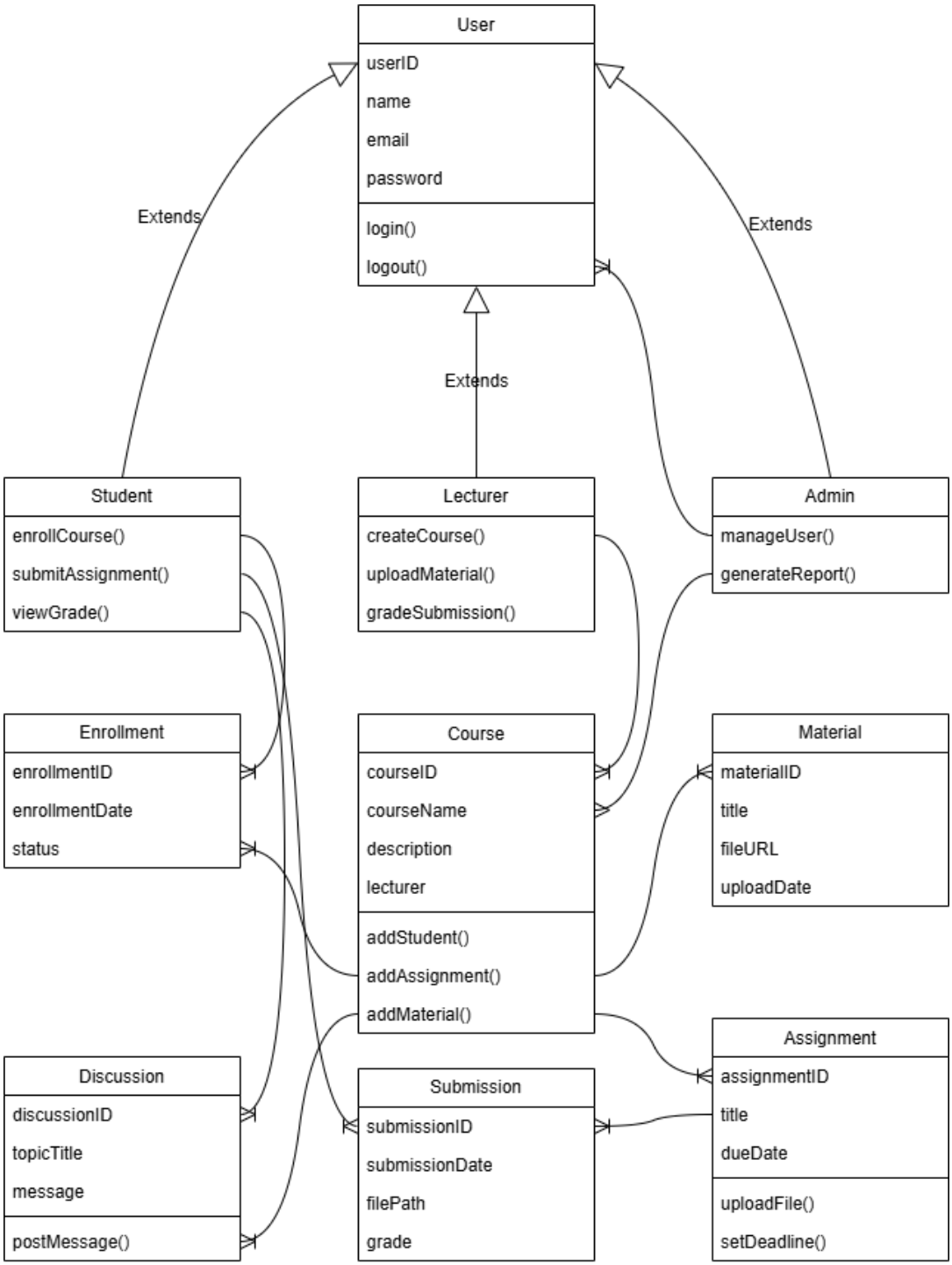
Lecturer interactions:

- Creates and manages courses.
- Uploads learning materials.
- Creates assignments.
- Grades student submissions.
- Posts announcements.

Admin interactions:

- Manages user accounts.
- Assigns lecturers to courses.
- Generates academic or usage reports.
- Maintains overall system control.

2.2 CLASS DIAGRAM



Object-Oriented Approach

- Used inheritance to promote reusability and reduce redundancy.

Clear Role Separation

Student, Lecturer, and Admin responsibilities are separated to enforce:

- Security
- Maintainability
- Scalability

Proper Relationship Modeling

- Association used for independent entities.
- Aggregation used when object belongs to another but can exist independently.
- Composition used when lifecycle dependency exists.

Extensibility

- Future roles (e.g., TeachingAssistant) can extend User easily.

Maintainability

- Modular class structure makes:
- Debugging easier
- Testing easier
- Feature addition safer

3.0 DOCUMENTATION FORMATTING

The SLMS documentation will follow the following format:

- Font: Times New Roman
- Font Size: 12
- Line spacing: 1.5
- Heading format:
    - 1.0 Main Heading (Bold)
    - 1.1 Subheading
- Alignment: Justified
- UML diagrams labelled properly
- Each section must include:
    - Purpose
    - Description
    - Example (if needed)

For code documentation, use:

- File header comment
- Function header comment
- Inline comments for complex logic

Example:

```
"""
File: course.py
Author: Team SLMS
Description: Handles course-related operations.
Date: 21 Feb 2026
"""
```

4.0 CODING FILE FORMATTING

- Indentation uses 4 spaces.
- Opening and closing brackets must align.
- Comments are required before functions.
- Files organised into folders.

Example:

```
if (loginSuccess) {
    openDashboard();
}
```

5.0 NAMING CONVENTIONS

Element	Convention	Example
Class	PascalCase	StudentAccount
Function	camelCase	submitAssignment()
Variable	snake_case	student_name
Constant	UPPER_CASE	MAX_LOGIN_ATTEMPT
File Name	lowercase	course.py

Term	Meaning
SLMS	Student Learning Management System
LMS	Learning Management System
Enrollment	Student registered in course
Submission	Assignment uploaded by student

6.0 FUNCTION AND METHODS MODULARITY

The system is designed using a modular approach where each function and method performs a single, well-defined task. Large functionalities are broken down into smaller and reusable functions to improve readability, maintainability, and scalability of the system.

For example, user authentication is separated into different functions such as input validation, credential verification, and session handling. This modular design allows easier debugging, testing, and future enhancement of the system.

Each module is designed to minimize dependency on other modules, allowing changes to be made with minimal impact on the overall system. This approach supports team collaboration, as different members can work on different modules independently.

7.0 CONTINUOUS INTEGRATION RULES

The following continuous integration rules are implemented to ensure effective collaboration and maintain code quality throughout the development of the Student Learning Management System (SLMS):

1. Each team member must develop features in a separate feature branch.
2. Direct commits to the main branch are prohibited.
3. All changes must be reviewed and approved by the team leader before merging.
4. Clear and meaningful commit messages are required for every commit.
5. Code must be tested to ensure it functions correctly before integration.
6. GitHub is used as the main platform for version control and collaboration.

These rules help reduce integration issues and improve the overall quality of the system.

Example:
Added assignment submission module.
Fixed login validation error.

8.0 REFERENCES

Moodle Official Website. https://moodle.org/

Moodle Codebase Repository. https://github.com/moodle/moodle